

# Week 10 - ML

|              |                 |
|--------------|-----------------|
| 📅 Start Date | @5 January 2026 |
| ☰ Weeks      | Week 10         |

## Predicting Continuous Target Variables With Regression Analysis

- **Regression Models** - Predict target variables on a continuous scale
  - Attractive for science questions
  - Other Examples:
    - Understanding relationship with variables
    - Evaluating trends
    - Making forecasts

## Linear Regression

- **Linear Regression** - Model relationship between one or multiple features and continuous variables
  - Predict outputs based on continuous scale

## Simple Linear Regression

- **Simple (Univariate) Linear Regression** - Model relationship between (explanatory variable,  $x$ ) and continuous target (response variable,  $y$ )

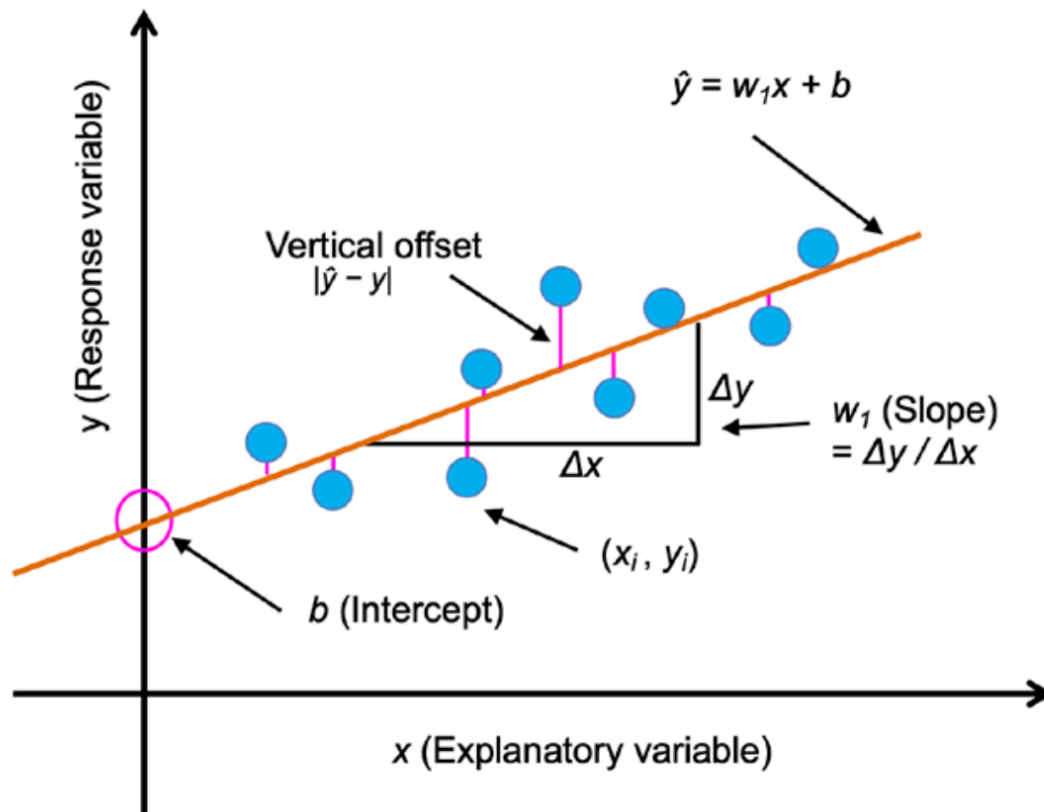
$$y = mx + b$$

Example used in book:

$$y = w_1x + b$$

- $b$  = Bias unit, represents  $y$  axis intercept
- $m(w_1)$  = Weight coefficient

- **Goal:** Learn weight of linear equation to describe relation between  $x, y \rightarrow$   
To predict new explanatory variables not part of training dataset

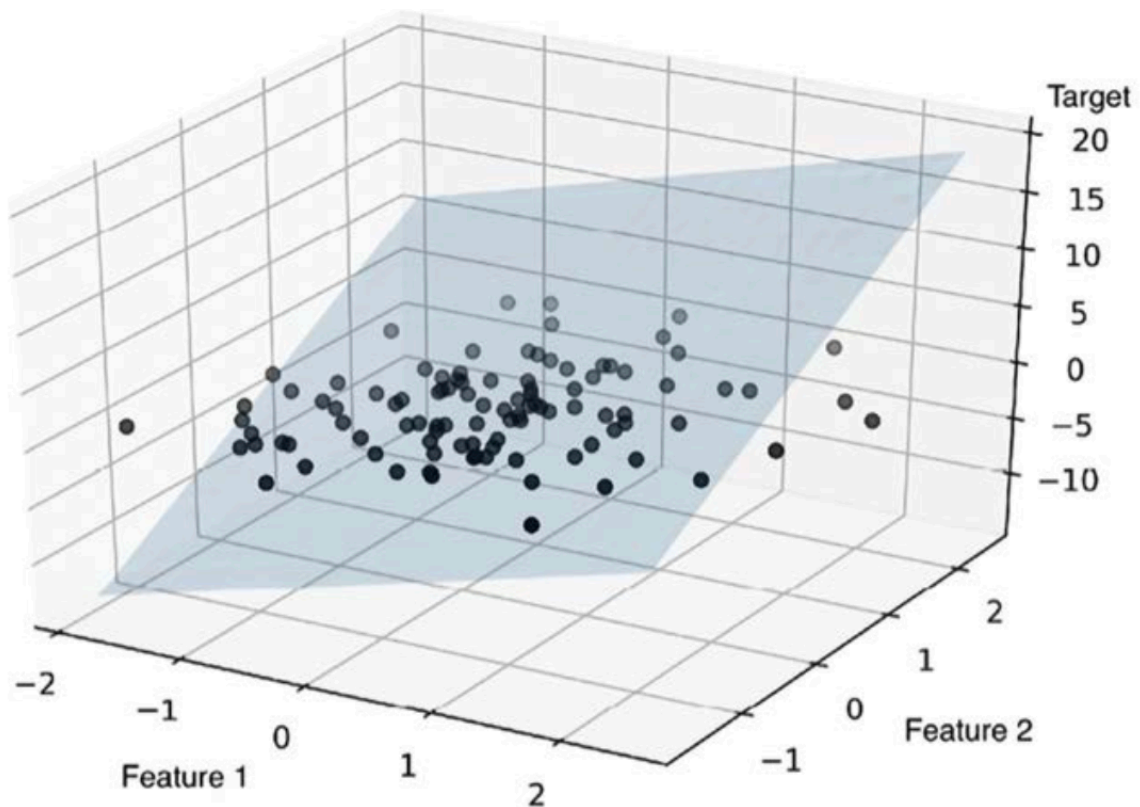


- Best fitting line  $\rightarrow$  **regression line**
- Vertical line off regression line, called **offset/residual** - **errors of predictions**

## Multiple Linear Regression

- **Multiple Linear Regression** - Generalised linear regression to multiple explanatory variables

$$y = w_1x_1 + \dots + w_mx_m = \sum_{i=1}^m w_ix_i + b = w_x^T + b$$



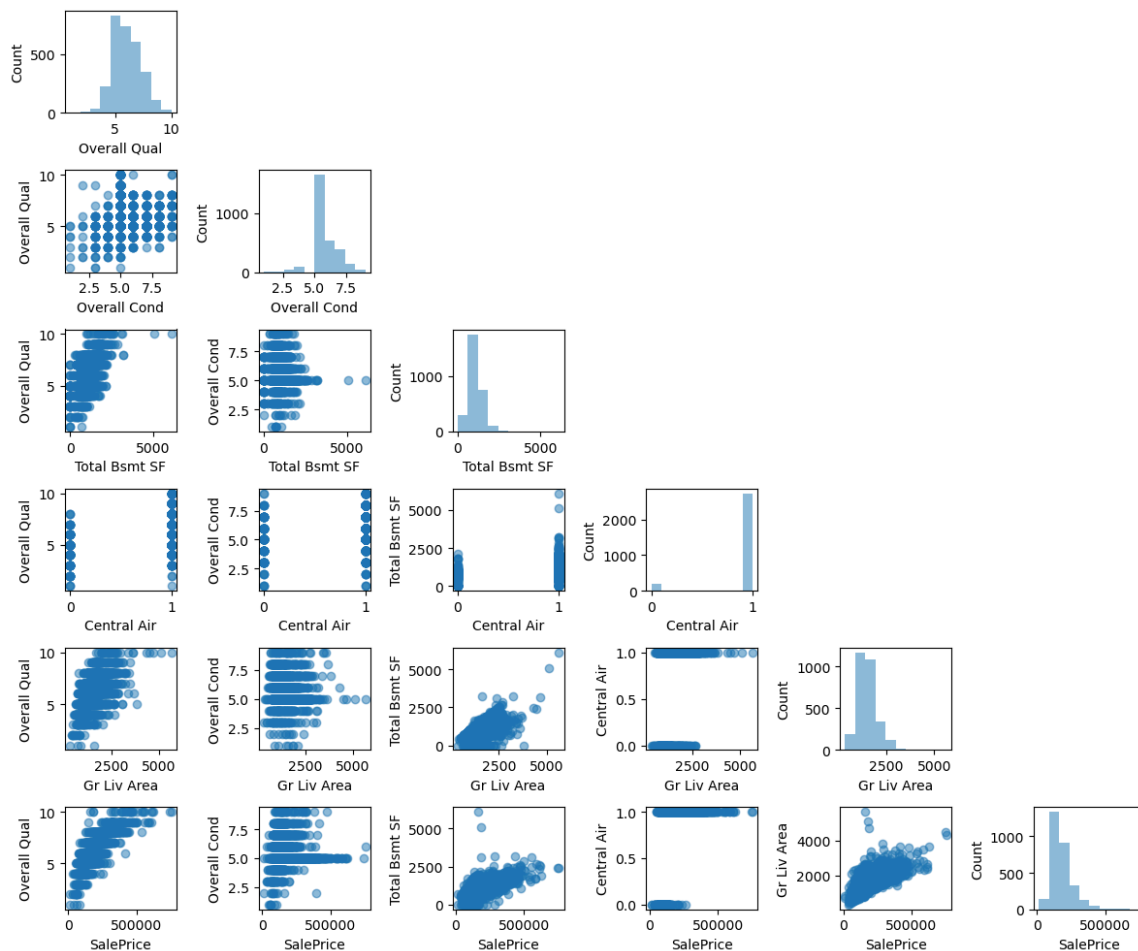
- Multiple linear regression, fits to dataset with three or more features



Simple and multiple linear regression → **based on the same concepts + same evaluation technique**

## Visualising Important Characteristics of a Dataset

- **Explanatory Data Analysis (EDA)** - Import + recommended first step before training machine learning model
- Scatterplot matrix → Visualise pair-wise correlation between different features in dataset in one place.



- **Scatterplot matrix** - quickly see how data is distributed and whether it contains outliers
- **Gr Liv Area** and **SalePrice** have a linear relationship



**Training a linear regression model, doesn't require normally distributed variables**

## Relationships Using Correlation Matrix

- **Correlation matrix** - Quantify and summarise linear relationships between variables
  - It's closely related to covariance matrix
- Correlation matrix is the rescaled version of the covariance matrix
- Correlation matrix = covariance matrix from standardised features

- Contains **Pearson's Product-moment Correlation Coefficient (Pearson's R)**  
 → Measures linear dependance between pairs of features.
  - Range is from -1 to 1
  - $r = 1$  and  $r = 0$  are perfect positive correlation and  $r = -1$  is the perfect negative correlation

$$r = \frac{\sum_{i=1}^n [(x^{(i)} - \mu_x)(y^{(i)} - \mu_y)]}{\sqrt{\sum_{i=1}^n (x^{(i)} - \mu_x)^2} \sqrt{\sum_{i=1}^n (y^{(i)} - \mu_y)^2}} = \frac{\sigma_{xy}}{\sigma_x \sigma_y}$$

- $\mu$  = Corresponding Feature
- $\sigma_{xy}$  = Covariance between  $x$  and  $y$
- $\sigma_x \sigma_y$  = Features' standard deviation

## Covariance vs. Correlation For Standardised Features

- Standardise feature  $x$  and  $y$ :

$$x' = \frac{x - \mu_x}{\sigma_x}, \quad y' = \frac{y - \mu_y}{\sigma_y}$$

- Compute (population) covariance between features:

$$\sigma_{xy} = \frac{1}{n} \sum_i^n (x^{(i)} - \mu_x)(y^{(i)} - \mu_y)$$

- Standardised features = **Mean zero** → calculate covariance:

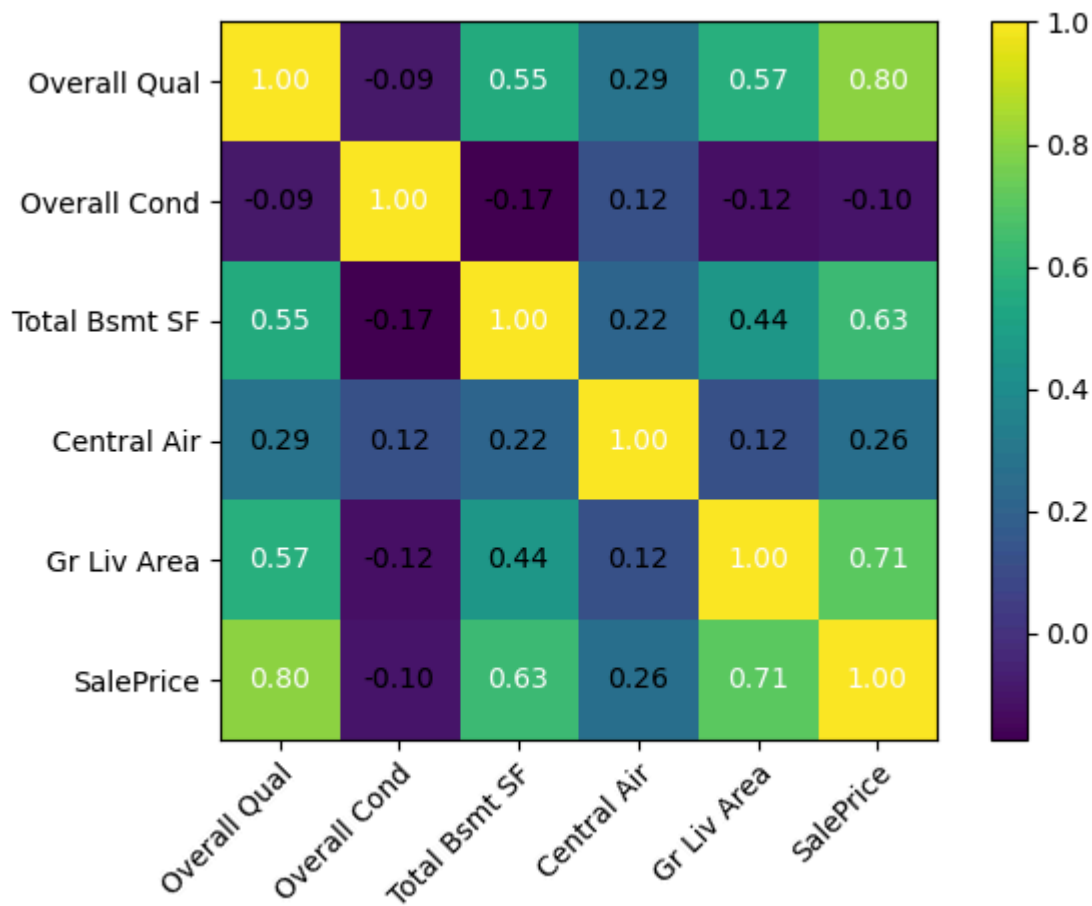
$$\sigma'_{xy} = \frac{1}{n} \sum_i^n (x'^{(i)} - 0)(y'(i) - 0)$$

- Resubstitution:

$$\begin{aligned} \sigma'_{xy} &= \frac{1}{n} \sum_i^n \left( \frac{x - \mu_x}{\sigma_x} \right) \left( \frac{y - \mu_y}{\sigma_y} \right) \\ \sigma'_{xy} &= \frac{1}{n \cdot \sigma_x \sigma_y} \sum_i^n (x^{(i)} - \mu_x)(y^{(i)} - \mu_y) \end{aligned}$$

- Simplify:

$$\sigma'_{xy} = \frac{\sigma_{xy}}{\sigma_x \sigma_y}$$



- Gives us information about linear correlation

## Ordinary Least Squares Linear Regression Model

- **Ordinary Least Squares (OLS)** - Sometimes called **Linear Least Squares** - Estimate parameters of linear regression line to minimise errors

## Regression Parameters with SGD

- Loss function of adaline = Mean Square Error (MSE) → **It's the same for OLS**

$$L(w, b) = \frac{1}{2n} \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2$$

- $\hat{y}$  = Predicted value  $\hat{y} = w^T x + b$
- **OLS** → **Can be understood as adaline without threshold function** → So we can get continuous value and not class label (0, 1)
- Closed-form solution for solving OLS:

$$w = (X^T X)^{-1} X^T y$$

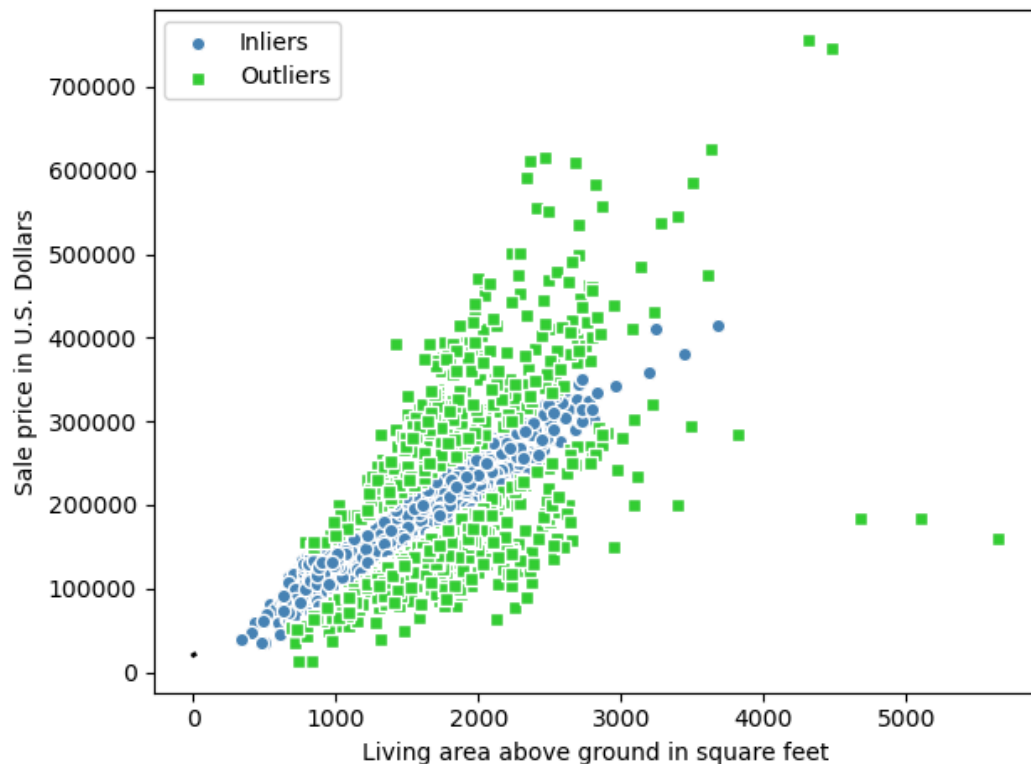
## Robust Regression Model Using RANSAC



Linear Regression Models can be heavily impacted by outliers

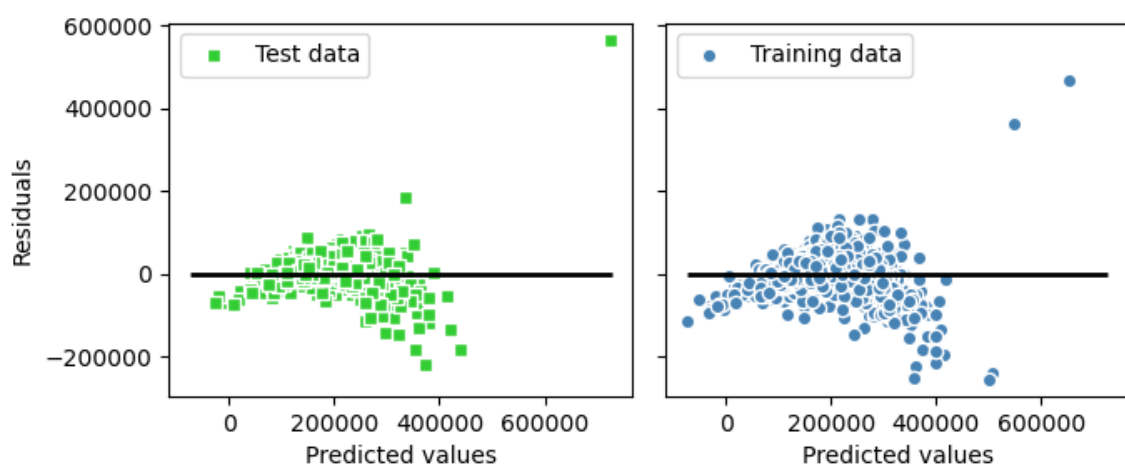
- Small subset can have a big effect on coefficient

- **RANdom SAMple Consensus (RANSAC)** - Fits a regression model to subset, called **inliers**
- Summarised Algorithm:
  1. Select random number of examples to be inliers and fit the model
  2. Test data points against fitted model and add points that fall within user-given tolerance to inliers
  3. Refit model with inliers
  4. Estimate error of fitted model vs. inliers
  5. Terminate algo if performance meets certain user-defined threshold, or fixed iteration number reached
- Using RANSAC we reduce potential outliers. **Only issue is that we don't know whether it is positive or negative**



## Performance Evaluation of Linear Regression models

- First we want to split dataset to train/test split
- **Residual plots** - Commonly used graphical tool for regression models
  - They can detect nonlinearity and outliers and check whether errors are randomly distributed



- Perfect prediction is,  $\text{Residual} = 0$  (this would never happen)



- **Good regression model** → Errors randomly distributed and residuals to scattered around centre line
- Patterns in residual model - **not good** - model unable to capture some explanatory information



Residual plots also good for detecting outliers

- **MSE (Mean Squared Error) algorithm good for measuring model performance** → Loss function to fit linear regression model

$$MAE = \frac{1}{n} \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2$$

- Without the 1/2 scaling factor. This would be used for simplifying derivative in gradient descent
- MSE → Normalises according to sample size,  $n$  → **Makes it possible to compare across different sample sizes**

$$MAE = \frac{1}{n} \sum_{i=1}^n |y^{(i)} - \hat{y}^{(i)}|$$

- MAE used because more intuitive to show error on original unit scale. This emphasises incorrect predictions slightly less
- **MSE/MAE for comparing models** → **unbound in contrast classification accuracy** → MSE/MAE depend on dataset and feature scaling



**Sometimes more useful to report coefficient of determination ( $R^2$ )**  
→ **Standardised version of MSE (better interpretability of model's performance)**

- $R^2$  = Fraction of response variance

$$R^2 = 1 - \frac{SSE}{SST}$$

- SSE = Sum of squared errors → similar to MSE, without normalisation by  $n$

$$SSE = \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2$$

- SST = Total sum of squares → **variance of response**

$$SST = \sum_{i=1}^n (y^{(i)} - \mu_y)^2$$

- $R^2$  → is a rescaled versino of MSE:

$$\begin{aligned} R^2 &= 1 - \frac{SSE}{SST} \\ &= \frac{\frac{1}{n} \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2}{\frac{1}{n} \sum_{i=1}^n (y^{(i)} - \mu_y)^2} \\ &= 1 - \frac{MSE}{var(y)} \end{aligned}$$

- $R^2$ , bounded between 0, 1 → **Can become negative for test dataset**
- Negative  $R^2$  = Regression model fits data worse than horizontal line (sample mean)



$R^2 = 1, MSE = 0$  Is the perfect fit

## Regularised Methods For Regression

- **Regularisation** - Tackling overfitting, by adding additional information → Shrinks parameter values of model
- **Most popular regularised linear regression:**
  - Ridge Regression
  - Least Absolute Shrinkage and Selection Operator (LASSO)
  - Elastic Net

### Ridge Regression

- L2 penalised model → Add squared sum of weights to MSE loss function:M

$$L(w)_{\text{Ridge}} = \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2 + \lambda ||w||_2^2$$

- Where the L2 term is:

$$\lambda ||w||_2^2 = \lambda \sum_{j=1}^m w_j^2$$

- $\lambda$  - Hyperparameter value → Increases regularisation strength, shrinks weight

## Lasso

- Can lead to sparse models → Depending on regularisation strength, weight can become zero → Useful for supervised feature selection

$$L(w)_{\text{Lasso}} = \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2 + \lambda ||w||_1$$

- Where the L1 term is:

$$\lambda ||w||_1 = \lambda \sum_{j=1}^m |w_j|$$

- Sum of absolute magnitude of model weight
- Limitation of Lasso → Selects at most  $n$  features if  $m > n$ , where  $n$  = Numer of training examples



**Property of lasso → advantage - Avoids saturated models**

- **Saturated model** → Number of training examples = number of features → Form of overparameterisation → Training data always fits perfectly.

## Elastic Net

- L1 penalty to generate sparsity and L2 penalty (selects more than  $n$  features if  $m > n$ )

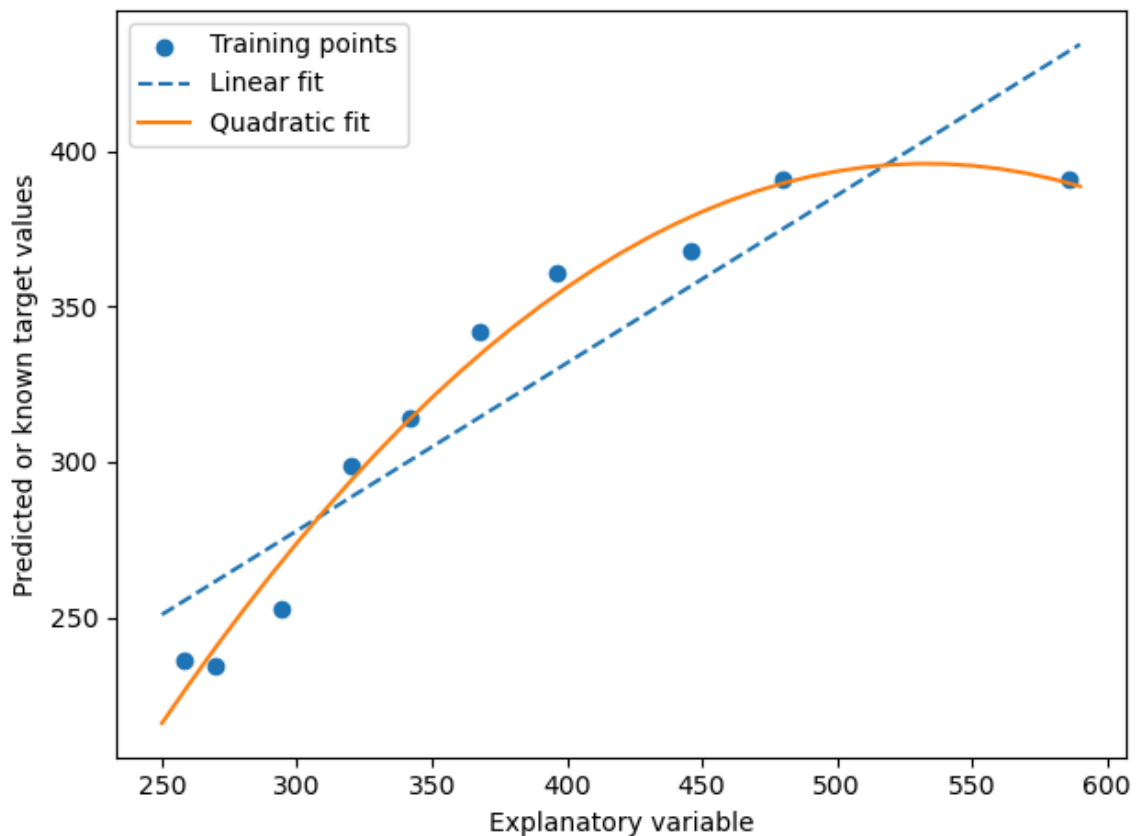
$$L(w)_{\text{Elastic Net}} = \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2 + \lambda_2 \|w\|_2^2 + \lambda_1 \|w\|_1$$

## Linear Regression Model To Curve-Polynomial Regression

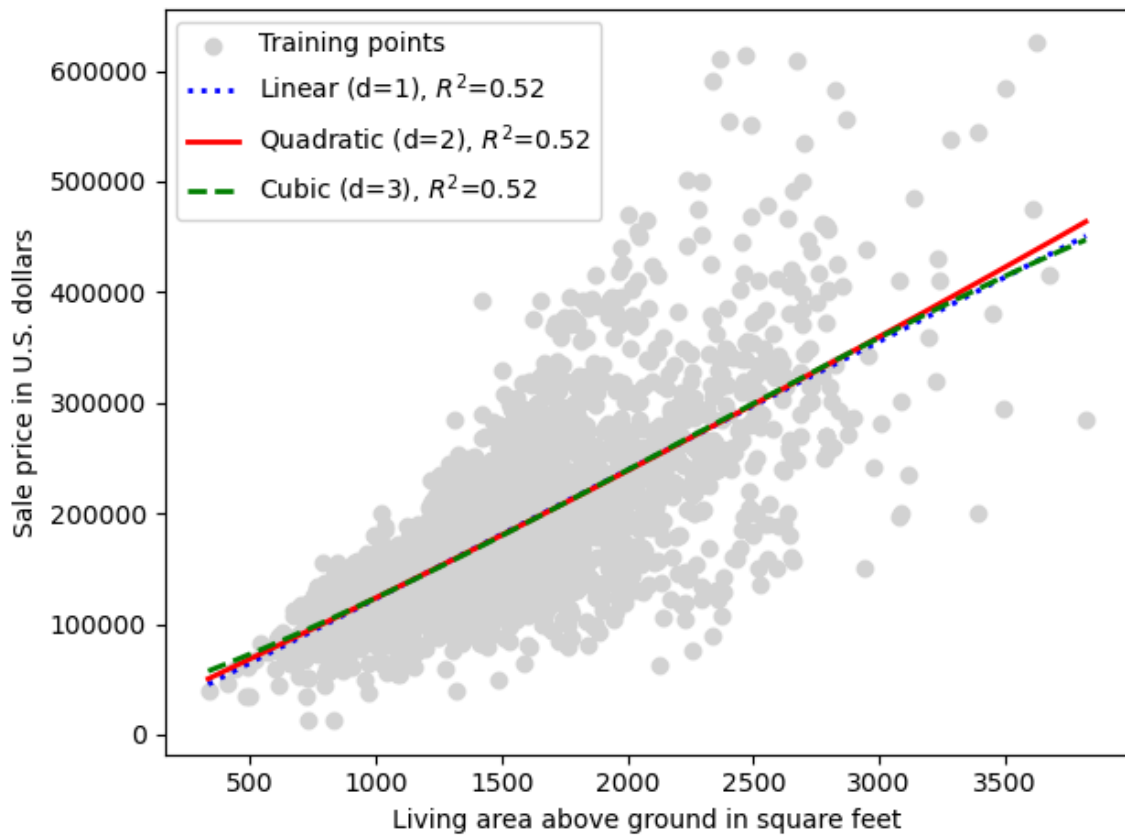
- Linear and Polynomial terms:

$$y = w_1x + w_2x^2 + \dots + w_dx^d + b$$

- $d$  = Degree of polynomial
- Polynomial regression is multiple linear regression, because of  $w$



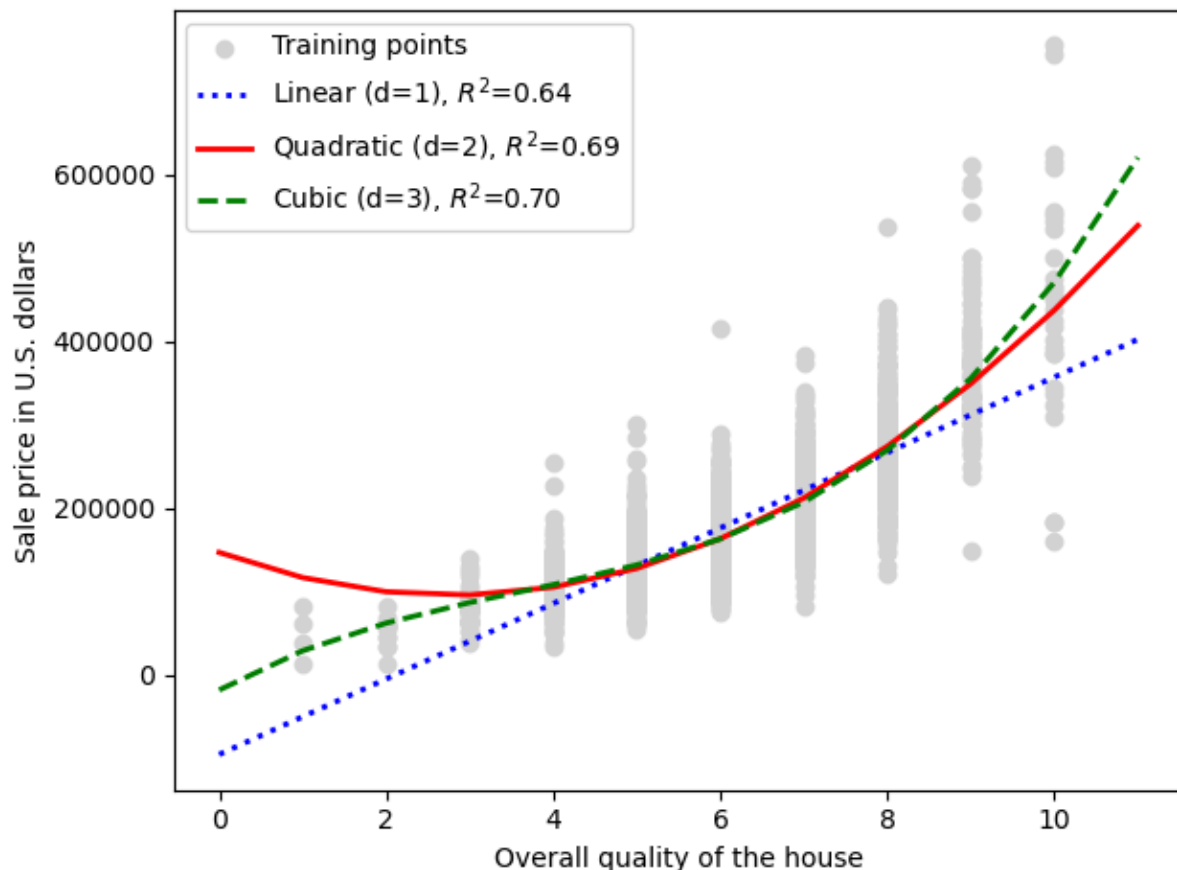
Comparison of linear vs. quadratic model



Comparison of different curves fitted to Sale Price/Living Area data



**Quadratic and cubic don't have effect, because the relationship is linear**



Linear, quadratic and cubic fits on data

- Quadratic and cubic better capture the relationship here
- Adding more polynomial features → Increases complexity → **Increases overfitting changes**
- In practice → Test on separate test dataset

## Non-Linear Relationships Using Random Forests

### Decision Tree Regression

- **Advantage is that it works with arbitrary features and doesn't require any transformation of features, if dealing with non-linear data**
- Defined entropy as measure of impurity to determine which feature split maximises **Information Gain (IG)**:

$$IG(D_p, x_i) = I(D_p) - \frac{N_{\text{left}}}{N_p} I(D_{\text{left}}) - \frac{N_{\text{right}}}{N_p} I(D_{\text{right}})$$

- $x_i$  = Feature that performs split
- $N_p$  = Number of training examples in parent node
- $I$  = Impurity function
- $D_p$  = Subset of training examples at parent node
- $D_{\text{left}}, D_{\text{right}}$  = Subsets of training examples after split
- Goal: Find feature split that maximises information gain
- Impurity metric for continuous variables → impurity measure of a node,  $t$ , as MSE:

$$I(t) = \text{MSE}(t) = \frac{1}{N_t} \sum_{i \in D_t} (y^{(i)} - \hat{y}_t)^2$$

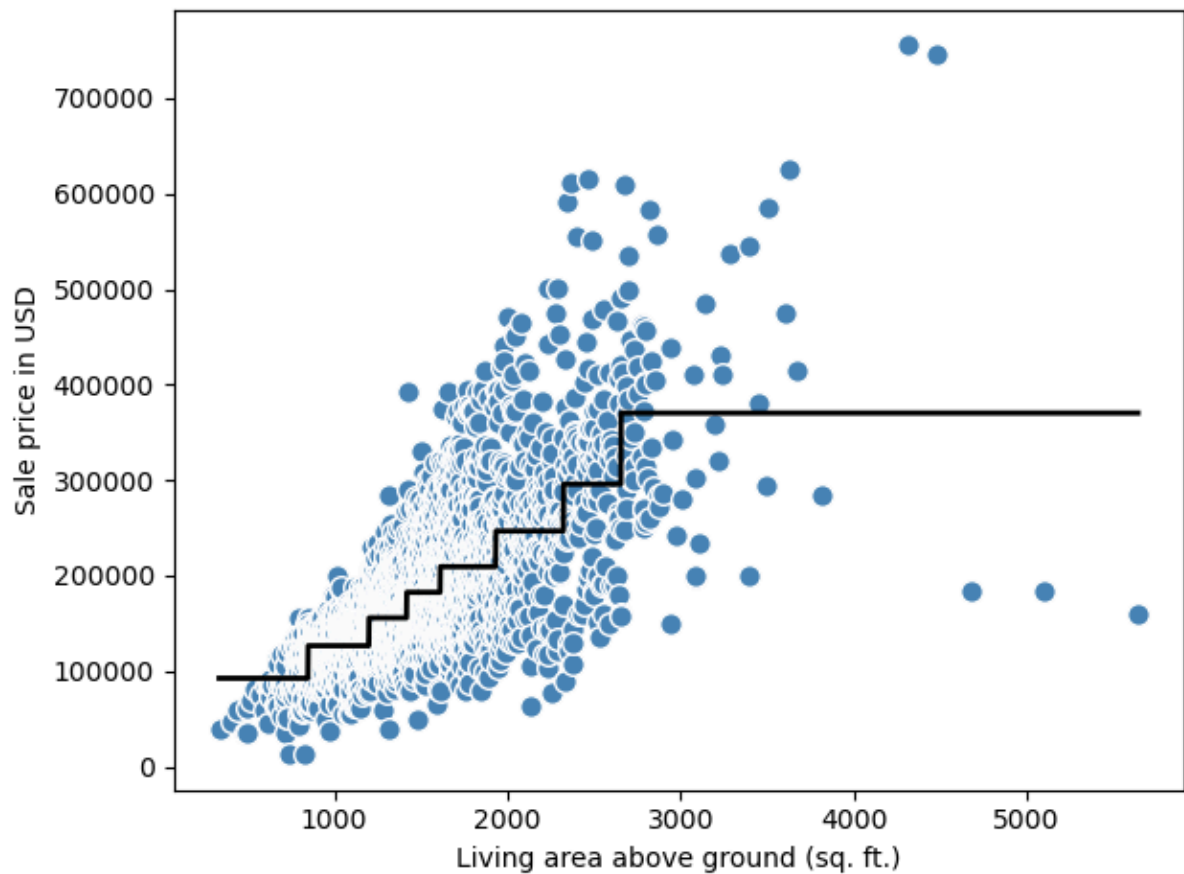
- $N_t$  = Number of training examples at node,  $t$
- $D_t$  = Training subset at node,  $t$
- $y^{(i)}$  = True target value
- $\hat{y}_t$  = Predicted value

$$\hat{y}_t = \frac{1}{N_t} \sum_{i \in D_t} y^{(i)}$$

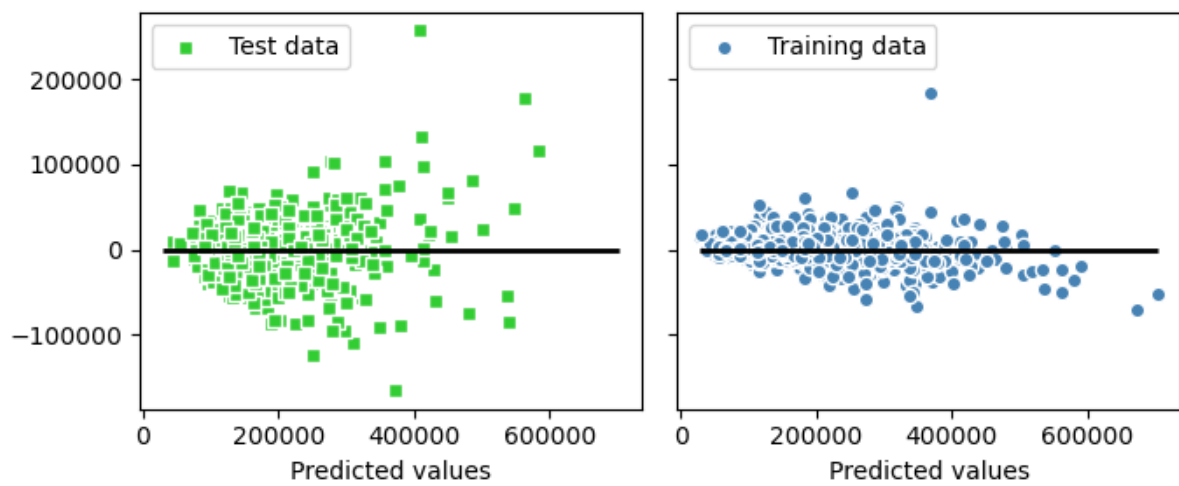
- Context of decision trees → MSE referred as within-node variance → Splitting criterion better known as variance reduction

## Random Forest Regression

- Better generalisation performance than individual decision tree because of randomness → Helps decrease model's variance
- Less sensitive to outliers and doesn't require hyperparameter tuning. The only tuning required is the number of trees in ensemble
- Random Forest algorithm - basically the same as basic random forest algorithm
  - Difference = MSE used to grow individual decision trees and predict target variables calculated as average prediction across all decision trees



A decision tree regression plot



The residuals of the random forest regression